Can a Model Tell Cause from Correlation? A Simple Pipeline on the Corr2Cause Dataset

Chaitanya Sai Meka, Rakshita, Dhanvin Vadlamudi, Kasula Lalithendra, Jagruthi Pulumati
CSAI Project, Phase 1 (Machine Learning)
Code: https://github.com/Jag2007/AI_ML_Project

***Abstract*—"Two things move together" does not mean "one causes the other". The Corr2Cause dataset tests exactly this. Each example gives a short text listing which variables are related, and a claim such as "A directly causes B". The task is to say whether the claim is definitely true (1) or not (0). We solved it in five steps, one notebook per team member: clean the data, turn the text into a cause-and-effect graph, build features, train classifiers, and finally draw the predicted graph and ask it "what if" questions. Using only the words of the text, our model scores an F1 of 0.47. Adding the graph-based features raises this to 0.80. Our final model, a Random Forest, scores an F1 of 0.905 on the unseen test set. 89% of the cause arrows it draws are correct, and the "what if" answers from our graphs match the true graphs in 5 out of 5 cases.**

I. Introduction

The problem. Ice cream sales and drowning cases both go up in summer. They are *correlated*, but ice cream does not cause drowning; hot weather causes both. Large language models often make this mistake and read correlation as causation.

The dataset. Corr2Cause [1] turns this into a yes/no task. Here is a real example (shortened):

> *Premise:* There is a closed system of 3 variables, A, B and C. A correlates with B. A is independent of C. B is independent of C.
> *Hypothesis:* There is a common effect (collider) of A and C.
> *Label:* 0 (not valid)

The *premise* lists which pairs of variables are related ("correlate") and which are not ("independent"). The *hypothesis* is a claim about cause and effect. The label is 1 only if the claim is true in *every* cause-and-effect picture that fits the premise. If even one fitting picture breaks the claim, the label is 0.

Our goal. Instead of a black box, we wanted a pipeline where every prediction can be traced back to a graph we can draw and explain. So we combined a classic causal method (the PC algorithm) with simple machine learning models.

II. Background and Related Work

Causal graph (DAG). We draw each variable as a node and each "X causes Y" as an arrow X→Y. Arrows cannot form a loop, so the picture is a *Directed Acyclic Graph* (DAG). Pearl [2] built the theory for reasoning with such graphs.

PC algorithm. PC [3] rebuilds a causal graph from "who is independent of whom". It starts with every pair connected, deletes the links between independent pairs, and then decides arrow directions where it can, using a few fixed rules [4]. Some arrows can still point either way, so PC returns a graph with some undirected links.

Equivalence class. Often several DAGs fit the same independence facts equally well [5]. This group of DAGs is called a *Markov equivalence class* (MEC). A Corr2Cause claim is valid only if it holds in all DAGs of the group.

Earlier work. Jin et al. [1] created Corr2Cause and showed that even fine-tuned language models do poorly on it. Kıcıman et al. [6] found similar weak spots in LLM causal reasoning.

Tools we used. TF-IDF [7] for text features; logistic regression, Random Forest [8] and gradient boosting [9] from scikit-learn [10]; the bootstrap [11] for error bars; and pgmpy [12] for PC and Bayesian networks.

III. Step 1: Cleaning the Data (Notebook 01)

We used the official train, validation and test splits from Hugging Face (Table I). Each row has the text (`input`), the `label`, the number of variables, and a `template` that says which type of claim it is.

Checking the data. We looked at the first and last rows, the shape and the column types. We counted missing values and duplicate rows. There were none, so nothing had to be filled in or removed.

Cleaning. Every text has one "Hypothesis:" marker, so we split it into two columns: `premise` and `hypothesis`. We also removed extra spaces. The same premise is reused with up to 90 different claims, so we gave each premise an ID number.

Removing leakage. A few premises appeared in two splits at once (2 in train and validation, 3 in validation and test). If the model sees a premise during training and again in testing, its score looks better than it really is. We left the test set as it was and removed these shared premises from train and validation.

Making train smaller. Train has about 206,000 rows, which is slow to work with. We randomly picked 400 premises (seed 42) and kept all of their claims, giving 35,340 rows. The share of valid claims stayed the same (18.5%).

IV. Looking at the Data (EDA)

Few valid claims. Only 13–19% of claims are valid (Fig. 1, left). A lazy model that always answers "not valid" would already be over 80% accurate. So accuracy is misleading here. Instead we use *precision* (of the claims we called valid, how many really were), *recall* (of the truly valid claims, how many

TABLE I
NUMBER OF ROWS BEFORE AND AFTER CLEANING

Split	Before	After	Premises
Train (sampled)	205,734	35,340	400
Validation	1,076	1,026	19
Test	1,162	1,162	21

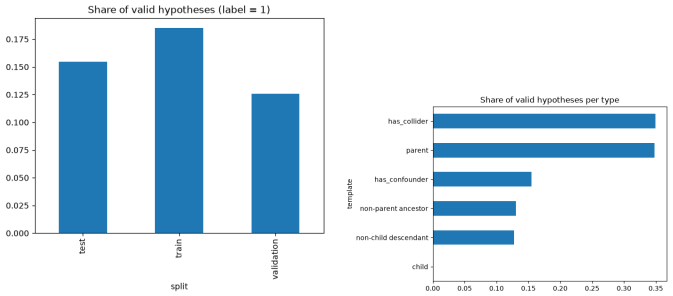


Fig. 1. Left: only 13–19% of claims are valid in each split. Right: share of valid claims for each claim type in train. "Child" claims are never valid.

we found) and *F1* (one number that balances the two). We also told the models to give more weight to the rare valid class.

A hidden shortcut. "Child" claims are never valid (Fig. 1, right), yet their text ("X directly causes Y") looks exactly like "parent" claims. So the `template` column gives away the answer. We did not use it as a feature; we only used it later to study mistakes.

Harder test set. Train only has graphs with 5–6 variables, but validation and test also have graphs with 2–4 variables. So the test set includes some cases the model never saw in training.

V. STEP 2: TEXT TO GRAPH (NOTEBOOK 02)

Reading the text. The sentences follow fixed patterns, so simple regular expressions are enough. From each claim we pick out the two variables (single capital letters) and the claim type (direct cause, indirect cause, common effect, common cause) using keywords. We also added safety checks: a variable must be one of A–F and must appear in the premise, a variable cannot cause itself (A→A), and unknown wording is rejected. All real rows passed, and broken examples we made by hand were caught.

Running PC. PC normally needs numeric data to test for independence. Here the premise already tells us which pairs are independent, so we let pgmpy's PC read those facts directly (`ci_test="independence_match"`). PC then (1) starts with all pairs linked and removes links between independent pairs, (2) finds patterns like X→Z←Y, where Z is a common effect, and (3) applies its fixed rules to set more directions. Links it cannot decide are left undirected (Fig. 2).

Counting fitting graphs. For each undirected link we try both directions. We keep every version that has no loops and fits the same independence facts. This gives us all the DAGs

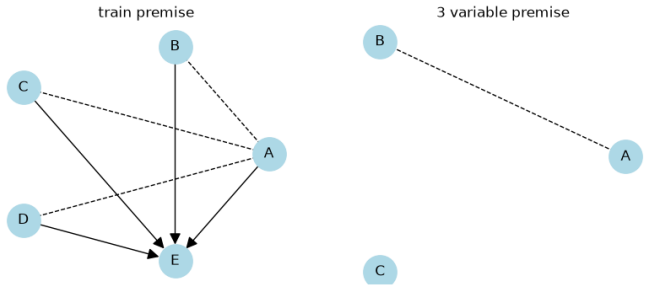


Fig. 2. PC output for two premises. Solid arrows have a known direction; dashed lines could point either way. Left: E is a common effect of A–D. Right: only A and B are linked, and C stands alone.

in the equivalence class, between 1 and 144 per premise. For each claim we then compute one number:

$$\text{mec_support} = \frac{\text{fitting DAGs where the claim is true}}{\text{all fitting DAGs}} \quad (1)$$

If mec_support is 1, the claim holds in every fitting graph, so it should be valid. This rule alone matches 93.7% of train labels, 96.5% of validation labels and 96.2% of test labels, which shows the text-to-graph step works. The few mismatches are cases where the dataset's label does not agree with what PC concludes.

VI. STEP 3: BUILDING FEATURES (NOTEBOOK 03)

A model cannot read text or graphs directly, so we turn each claim into a row of numbers called features.

Graph features (15). The claim type (one column per type); how many correlations and independences the premise lists; whether X and Y are said to correlate, to be independent, or to be independent once other variables are known; whether PC drew X→Y, Y→X or an undirected X–Y; the number of fitting DAGs (on a log scale); and mec_support. We rescaled these to a similar range, using only train data to set the scale.

Text features. TF-IDF gives each word or word pair a score: high if it appears often in this text but rarely in others. We made one TF-IDF table for the premise (135 terms) and one for the claim (106 terms), learned from train only. We kept one-letter words because A–F are the variable names.

Which features matter? We scored each feature by how much it tells us about the label (mutual information, Fig. 3). mec_support is far ahead of everything else, followed by the arrow-direction features. The top text words are things like "collider" and variable letters, which say nothing about the specific pair being asked about. Keeping only the top 80 text terms barely changed the score (F1 0.80 vs 0.82), so we kept all of them.

Text vs graph features. Table II trains the same simple model (logistic regression) on three feature sets. With text only, the model calls almost everything valid: it finds 95% of valid claims, but only 31% of its "valid" answers are right. The graph features fix this, and using both is best.

Two different models agree (Fig. 4): the graph features, not the words, drive the predictions.

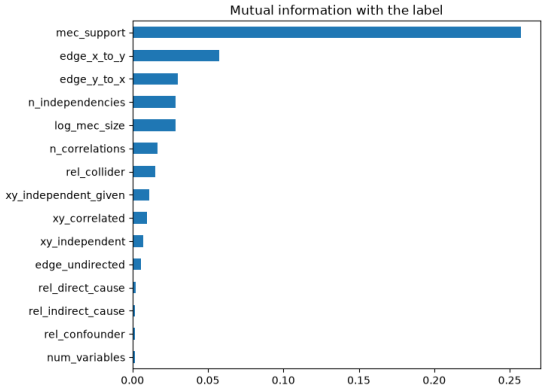


Fig. 3. How much each graph feature tells us about the label (train). mec_support, which comes from PC, is clearly the most useful.

TABLE II
WHICH FEATURES HELP? (LOGISTIC REGRESSION, VALIDATION SET)

Features	Precision	Recall	F1
Text only	0.311	0.953	0.469
Graph only	0.634	0.915	0.749
Text + graph	0.695	0.938	**0.799**

VII. STEP 4: TRAINING AND TESTING MODELS (NOTEBOOK 04)

Setup. Each claim becomes 256 numbers (text + graph features). We tried three models: logistic regression, Random Forest (300 decision trees voting together) and gradient boosting (small trees added one after another, each fixing the last one's mistakes). For each model we tried a few settings with `GridSearchCV` and 5-fold cross-validation.

One detail matters here: we used `GroupKFold`, which keeps all claims of the same premise in the same fold. Otherwise the model could see a premise in training and be checked on the same premise, which would make the score look too good. We picked the model with the best validation F1, and used the test set only once, at the very end.

Results. Random Forest had the best validation F1, so we chose it (Table III). Gradient boosting does a bit better on test, but switching models after seeing test scores would be cheating, so we kept our choice. Random Forest also beats the plain mec_support rule (0.905 vs 0.871), mostly because it finds more of the valid claims.

How sure are we? The test set has only 21 premises, so the score could change with a different test set. To check, we re-sampled whole premises 1,000 times (the bootstrap) and recomputed F1 each time. The F1 stayed between 0.852 and 0.937 in 95% of the runs.

Where it goes wrong. The model made 36 mistakes out of 1,162 (Fig. 5). Most are false alarms: 14 on common-effect claims and 8 on "parent" claims. These are the same cases where PC and the dataset labels disagree in Step 2. It made no mistakes on "child" claims. Graphs with 2–4 variables, which never appeared in training, had about the same error rate (4.0%) as 5–6 variable graphs (3.0%). This is because the graph features come from the parser and PC, not from memorised sentences.

TABLE III
RESULTS (VALIDATION F1; TEST PRECISION, RECALL AND F1)

Model	Val F1	Test P	Test R	Test F1
Logistic Regression	0.784	0.783	0.922	0.847
Random Forest (chosen)	**0.855**	0.860	0.956	**0.905**
Gradient Boosting	0.843	0.907	0.972	0.938
mec_support rule only (no ML)	–	0.920	0.828	0.871

VIII. STEP 5: DRAWING THE GRAPH AND ASKING "WHAT IF" (NOTEBOOK 05)

Building the DAG. For each test premise, every "X directly causes Y" claim that the model calls valid becomes an arrow. We add arrows starting with the ones the model is most sure about. An arrow is skipped if it points from a variable to itself, uses a variable not in the premise, or would create a loop. All 21 test graphs came out as proper DAGs. Out of 73 arrows drawn, 65 are correct (**precision 0.89**), and no true arrow was missed (recall 1.00). All 8 wrong arrows are also drawn by PC, so they come from the same label disagreement as before (Fig. 6).

Bayesian network and "what if" questions. A Bayesian network is a DAG with probabilities attached to each node. It lets us ask "what happens to Y if we *force* X to a value?", written $do(X)$. This is different from just observing X, and it is the real test of cause and effect.

Corr2Cause has no numbers, only text, so we made some. We gave the true graph some probabilities, generated 5,000 samples from it, and fitted our predicted DAG on those samples with pgmpy. Then we measured the effect of X on Y as

$$P(Y{=}1 \mid do(X{=}1)) - P(Y{=}1 \mid do(X{=}0)). \qquad (2)$$

For the chain A→B→C, the true effect of A on C is 0.490 and our correctly drawn graph gives 0.498. If the arrows are reversed, the graph still fits the same data but gives 0.000, and wrongly says that C affects A. On the five test premises whose graph is fully known, all five of our answers were within 0.05 of the true answer. So getting the arrow directions right is what makes the "what if" answers right.

IX. CONCLUSION AND NEXT STEPS

Turning the text into a graph with a parser and the PC algorithm already explains 94–96% of the labels. Adding these graph features to simple text features and training a Random Forest gives a test F1 of 0.905, with no data leaking between splits. Most remaining mistakes come from a small set of labels that disagree with PC. In Phase 2 we plan to replace TF-IDF with a fine-tuned transformer that reads the text better, combine it with the PC features, and measure how much each part helps.

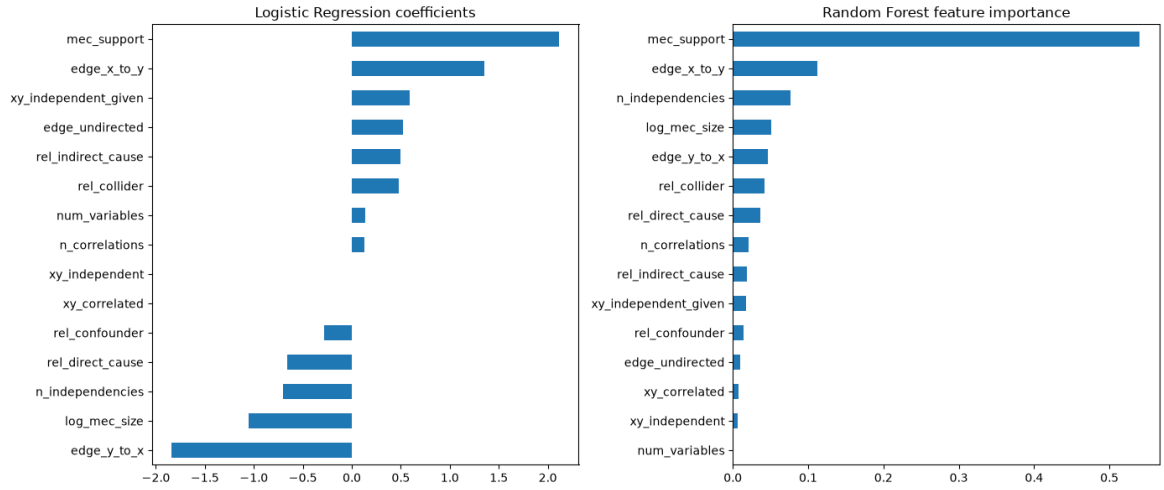


Fig. 4. Left: logistic regression weights. mec_support and an X→Y arrow push towards "valid"; a Y→X arrow pushes towards "not valid". Right: Random Forest importance, where mec_support alone takes 0.54 of the total.

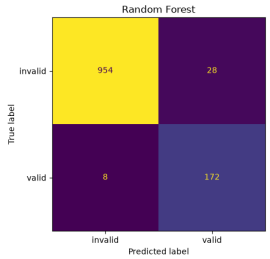


Fig. 5. Random Forest on test: 954 correct "not valid", 172 correct "valid", 28 false alarms and 8 missed valid claims.

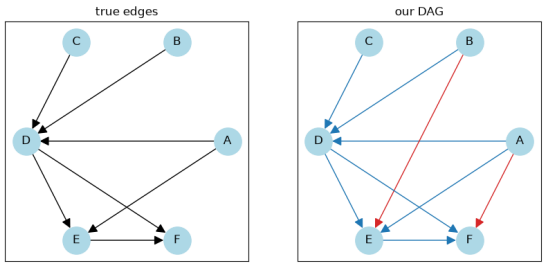


Fig. 6. One test premise. Left: the true arrows. Right: our predicted DAG. Blue arrows are correct; the two red arrows (B→E, A→F) are extra ones that PC supports but the dataset marks as not valid.

Contribution Matrix

Each person owned one notebook (✓ = owner).

Task	Chaitanya	Rakshita	Dhanvin	Lalithendra	Jagruthi
Loading and checking data	✓				
Cleaning, leakage, sampling	✓				
EDA and plots	✓				
Text parser and safety checks		✓			
PC algorithm, mec_support		✓			
Building features			✓		
Feature selection, text vs graph			✓		
Model tuning and selection				✓	
Evaluation, error bars, mistakes				✓	
Building the DAG, arrow precision					✓
Bayesian network, "what if"					✓
Report and repository					✓

References

[1] Z. Jin, J. Liu, Z. Lyu, S. Poff, M. Sachan, R. Mihalcea, M. Diab, and B. Schölkopf, "Can large language models infer causation from correlation?" in *Proc. ICLR*, 2024.

[2] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge Univ. Press, 2009.

[3] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*, 2nd ed. MIT Press, 2000.

[4] C. Meek, "Causal inference and causal explanation with background knowledge," in *Proc. UAI*, 1995, pp. 403–410.

[5] T. Verma and J. Pearl, "Equivalence and synthesis of causal models," in *Proc. UAI*, 1990, pp. 255–268.

[6] E. Kıcıman, R. Ness, A. Sharma, and C. Tan, "Causal reasoning and large language models: Opening a new frontier for causality," *arXiv:2305.00050*, 2023.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[8] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[12] A. Ankan and A. Panda, "pgmpy: Probabilistic graphical models using Python," in *Proc. SciPy*, 2015.